Day 1 – Environment & Project Setup

🏷️ Setup & Prerequisites

1. **Setup Checklist**:
   - **Install Python 3.12+**: Ensure Python 3.12 or higher is installed on your local machine. You can check your version by running `python3 --version` or `python --version` in your terminal.
   - **Prepare Your Workspace**: Create an empty project directory on your computer and open a clean terminal window pointing to this folder.
   - **Web Browser Ready**: Have a modern web browser open and ready to navigate to the local server address: `http://127.0.0.1:8000/`.

2. **Prerequisites**:
   - You must have a working terminal environment suitable for your operating system:
     - **macOS/Linux**: Standard Terminal or Zsh.
     - **Windows**: Git Bash or PowerShell (avoid using command prompt `cmd` as some unix-like commands will not be supported).

🎯 Learning Objectives

By the end of this class, you will be able to:

- Install and configure modern Python package management using `uv`.
- Initialize and activate a virtual environment.
- Scaffold a new Django project and custom app.
- Securely configure environment variables using `python-decouple`.
- Initialize a Git repository and prevent secrets leaks using `.gitignore`.
- Run the Django development server and verify the default setup.

⏱️ Classroom Timeline (45 min)

Segment	Duration	Focus
1. Hook & Big Picture	7 min	Interactive overview of the bootcamp outcome (game key expiry & publisher webhooks).

2. Key Concepts & Core Ideas	7 min	Introduction to MTV, API backends, package management (`uv`), and configuration security.
3. Live Coding Walkthrough	23 min	Live setup of python environment, Django, and decoupling configuration.
4. Check for Understanding	5 min	Q&A on virtual environments, secrets management, and DEBUG mode.
5. Class Wrap-up & Checkpoint	3 min	Verification of students' local environments.

🚀 Lecture Step-by-Step Delivery Plan

1. Hook & Big Picture (7 min)

The business problem: digital game key marketplaces need to deliver keys to users, track their expirations, and automatically notify publishers when a key expires so they can deactivate it on their servers.

- **Big Picture**: The finished flow: User buys key → key expires → Celery fires webhook → publisher receives it.
- **Q&A**:
  - **Question**: Why do we need webhooks instead of polling?
    - *Answer*: Polling is inefficient because clients have to repeatedly query the database (creating high traffic and latency). Webhooks are event-driven: our system pushes notifications only when an event happens, reducing server load and ensuring near-instant updates.
  - **Question**: Why is this a production-grade problem?
    - *Answer*: It involves cross-system communication, payload security (signing), concurrency (race conditions), reliability (retries, queues), and audit logging.

2. Key Concepts & Core Ideas (7 min)

Introduce the following definitions to you:

- **What is Django?** MTV (Model-Template-View) framework. Mention that we'll use it purely as a REST API backend with no HTML templates.
- **What is DRF?** Django REST Framework. A toolkit built on top of Django that gives us serializers, viewsets, and routers for building RESTful APIs.
- **Virtual environments**: Why isolation matters (preventing dependency conflicts and ensuring reproducibility across developers' machines).
  - *Note - Why do we isolate?* Without virtual environments, installing packages globally can overwrite

dependencies needed by other projects on the same machine, causing conflicts and breaking applications.

- **DEBUG=True vs False**: Django displays highly detailed error pages containing traceback information. Explain that this is crucial for development but a critical security leak in production.
  - *Note - Why is it a leak?* It exposes database queries, settings, system file paths, configurations, and environment variables to the end user.

- **Why python-decouple?** Reading from `os.environ.get()` is functional but `decouple` simplifies casting types (e.g. converting a string `"True"` to boolean `True`) and handling defaults and local `.env` files in a clean API.

---

3. Live Coding Walkthrough (23 min)

Step 3.1: Tooling Installation & Scaffolding

- **Concept**: Contrast standard `pip` + `venv` with `uv`. We use `uv` for speed, lockfiles, and reproducibility. `uv` is an extremely fast Python package installer and resolver written in Rust. It serves as a drop-in replacement for `pip`, `pip-tools`, and `virtualenv`. Run the installation and scaffolding commands.
- **Code**:

```
# Install uv (macOS/Linux)
# This command downloads the uv binary installation script and executes it via the shell.
curl -LsSf https://astral.sh/uv/install.sh | sh

# Create and activate virtual environment
# uv venv creates a local virtual environment named '.venv' in the current working directory
uv venv

# Activate the environment so your terminal uses the virtual environment's Python interprete
source .venv/bin/activate

# Add dependencies
# uv add automatically adds packages to your environment and manages a project lockfile.
# Packages installed:
# - django: The core web framework.
# - djangorestframework: Toolkit for building RESTful APIs on top of Django.
# - python-decouple: Utility for separating settings configuration from source code.
# - celery: Task runner for asynchronous and background execution.
# - redis: Python client for connecting to the Redis database (used as Celery's broker).
# - pytest-django: Django integration helper for testing with pytest.
uv add django djangorestframework python-decouple celery redis pytest-django

# Scaffold project and app
# django-admin startproject creates a folder structure containing core settings.
# The '.' at the end is CRITICAL: it creates the project in the CURRENT directory, preventin
django-admin startproject gamekey_platform .

# startapp creates a custom Django application named 'games' containing models, views, and c
```

```
python manage.py startapp games
```

- **Verify**: Confirm that you see `(.venv)` in your command line prompt. Ensure you are inside the active virtual environment before moving on. Run `which python` (or `where python` in Windows PowerShell) to verify it points to the `.venv` directory. ⚠ **Common Pitfalls I will flag**:
- **Forgetting to activate the venv**: If you skip `source .venv/bin/activate`, you will install dependencies globally or get "django not found". Run `which python` to verify the path points to `.venv`.
- **Windows path differences**: Note that on Windows, the activation command is `.venv\Scripts\activate` rather than `source .venv/bin/activate`.

Step 3.2: Environment Variable Configuration

- **Concept**: Secrets (such as the Django `SECRET_KEY` or database passwords) should never go into version control (source code). If you push a secret key to a public GitHub repository, bots will scan it within seconds and exploit it. We extract these values into a local environment file called `.env`.
- **Code**: Create `.env` in the project root directory:

```
SECRET_KEY=your-secret-key-here
DEBUG=True
```

- **Verify**: Check that the `.env` file is created and contains the correct key-value pairs.

Step 3.3: Django settings.py Decoupling

- **Concept**: Now we need to modify Django's settings to read our newly created `.env` file rather than having hardcoded variables. `python-decouple` provides a helper function `config()` that searches for the variable in the `.env` file first, and falls back to system environment variables if not found.
- **Code**: Update `gamekey_platform/settings.py` to import and use `python-decouple`:

```
# Add this import at the top of the file
from decouple import config

# Replace the hardcoded SECRET_KEY line with this:
SECRET_KEY = config('SECRET_KEY')

# Replace the hardcoded DEBUG line with this:
# The default is set to False for safety, and cast=bool converts the environment string "True"
DEBUG = config('DEBUG', default=False, cast=bool)
```

- **Verify**: Run the development server to verify the configuration is loaded correctly:

```
# Verify setup
python manage.py runserver
```

Navigate to and verify the Django welcome page loads. ⚠ **Common Pitfalls I will flag**:

- **`SECRET_KEY` still hardcoded**: You might add `python-decouple` but forget to update the actual variables in `settings.py`. Test this by removing the key from `.env` and seeing if Django throws an error on startup.

Step 3.4: Git Initialization

- **Concept**: Set up version control. Introduce the `.gitignore` file and emphasize that the `.env` file must be ignored immediately to prevent leaking the secret keys.
- **Code**:

```
git init
```

  Create `.gitignore` and include `.env` (along with standard Python/Django patterns: `__pycache__`, `.venv`, `*.pyc`, etc.):

```
.env
.venv/
*.pyc
__pycache__/
db.sqlite3
```

- **Verify**: Run `git status` and confirm that `.env` is NOT listed under untracked files. ⚠ **Common Pitfalls I will flag**:
- **Committing `.env` to Git**: Check what `git status` looks like *before* and *after* editing `.gitignore` to see how Git tracks files.

---

4. Check for Understanding (5 min)

> ***Question**: "Why can't we just hard-code `SECRET_KEY = 'abc123'` in settings.py?"*
> ***Answer**: Hard-coding `SECRET_KEY` in source code poses a severe security risk. If the repository is pushed to a public platform (like GitHub), anyone can see the key. Attackers can use it to forge signed cookies, session data, or password reset tokens. Storing it in an environment variable (`.env` file) keeps it private and allows different values for development and production environments.*

> ***Question**: "What would happen if two projects on the same machine installed different versions of `django` without virtual environments?"*
> ***Answer**: Installing different versions globally would cause a conflict because Python can only resolve one version of a package in its global site-packages directory. Installing a different version for the second project would overwrite the version needed by the first project, breaking it. Virtual environments isolate dependencies per project, allowing each to run its required version*

> *independently.*

> ***Question**: "What does `DEBUG=True` change about how Django behaves?"*
> ***Answer**: When `DEBUG=True`, Django displays detailed error traceback pages to the client (useful for debugging but a huge security leak in production because it exposes database queries, settings, and path variables). It also runs the built-in development server's auto-reloader and serves static/media files automatically. In production (`DEBUG=False`), it returns a generic 500 error page, requires `ALLOWED_HOSTS` to be set, and disables automated static file serving.*

---

5. Daily Checkpoint & Wrap-up (3 min)

- **Verification Criteria**:
  - ○ You can access the Django welcome rocket page running locally at `http://127.0.0.1:8000/`.
  - ○ The `.env` file exists and contains `SECRET_KEY` and `DEBUG`.
  - ○ Running `git status` shows `.env` is untracked (not staged) and excluded.
  - ○ Running `git log --oneline` shows at least one commit.